

Unidad 3. Tipos de Datos Estructurados

Uniones

Por definición, una variable es un segmento de memoria caracterizado por su dirección de inicio, extensión e interpretación. Una **union** es el análogo de una estructura, salvo por un detalle: todos los campos de que consta una estructura comienzan en una misma posición, que es la dirección de comienzo de la estructura. Consiguientemente, las uniones tienen una longitud que es precisamente la del mayor de sus campos. Dicho de otro modo, una unión es una estructura en que todos los campos están superpuestos, ocupando la misma porción de memoria física. Si algunos campos son más cortos y otros más largos, el aprovechamiento real de memoria puede no ser total.

Una union se declara con el mismo formato de una struct. Primero declaramos el tipo union y luego declaramos variables de ese tipo. Por ejemplo:

```
union nombre_union{
    int valorInt;
    float valorFloat;
    char valorChar[15];
};
```

Variables del tipo unión:

```
nombre_union dato1, *dato2; //dato1 es un valor estático y dato2 es un valor dinámico
```

La variable será lo suficientemente grande como para mantener el mayor de los tres tipos. Valores de cualquiera de esos tipos pueden ser asignados a dato1 y después empleado en expresiones mientras que el uso sea consistente: el tipo recuperado debe ser el tipo que se almacenó más recientemente.

*Es responsabilidad del programador llevar el registro del tipo que está almacenado actualmente en una union. El resultado de referenciar un miembro de union diferente al ultimo que se guardó es indefinido, ya que el dato almacenado se trata como de tipo diferente.

El acceso a los miembros de una union:

```
variable_union.miembro
```

por ejemplo:

```
dato1.valorFloat;
dato2->valorInt;
```

Las uniones pueden presentarse dentro de estructuras y arreglos y viceversa. Las operaciones que se pueden realizar sobre uniones son las siguientes: asignar una union a otra del mismo tipo, tomar la dirección de un tipo union, acceder a los miembros de una union utilizando el operador de miembro de estructura (.) o el operador de apuntador a estructura (&).

Una union puede inicializarse con un valor del mismo tipo que el primer miembro de la union.

*No podemos comparar uniones debido a que no se sabe cual miembro de cada una está activo.